

Міністерство освіти і науки України
Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Звіт
з лабораторної роботи No 5
з дисципліни
«Мова програмування Java»

Виконала:
студентка групи ЗПІ-зп41
Ірина Левківська

Перевірив:
Орленко С.П.

Київ 2025

Поставка завдання

1. Напишіть програму, яка отримає рядок з максимальною кількістю слів із заданого файлу.
2. Зашифрувати вхідний символьний потік за наступним алгоритмом: замінити кожний символ на такий, код якого більший на значення коду ключового символу (шифр Цезаря). При виконанні завдання:
 - a. Реалізувати метод, який реалізує шифрування
 - b. Реалізувати метод, який реалізує дешифрування.
 - c. Використати для розв'язання задачі потоки типу `FilterOutputStream-FilterInputStream/FilterWriter-FilteredReader`.
3. Підрахувати частоту, з якою зустрічаються теги на сторінці по URL (URL на ваш вибір).
 - a. Вивести результати наступним чином: в порядку зростання для тегів в лексикографічному порядку
 - b. Вивести результати наступним чином: в порядку зростання по частоті появи тегів

Вирішення завдання (Завдання 1)

При роботі над цією задачею я зосередилася на тому, щоб програма не просто знаходила один довгий рядок, а коректно обробляла вміст файлу, навіть якщо в ньому є кілька рядків з однаковою кількістю слів. Для зчитування даних я використала бібліотеку `java.nio.file`, що дозволяє отримати всі рядки файлу у вигляді списку одним методом. Весь алгоритм обробки я побудувала на базі `Stream API`, розділивши процес на два логічні етапи.

Спочатку програма аналізує кожен рядок, щоб обчислити максимальну кількість слів. Для цього я використовую метод `split("\\s+")`, який дозволяє правильно розбити текст, ігноруючи будь-яку кількість пробілів між словами. На цьому етапі також важливо було врахувати порожні рядки: я додала перевірку через `trim().isEmpty()`, щоб вони не викликали помилок і мали нульову кількість слів. Після того, як пікове значення знайдено, я проводжу другий етап — фільтрацію. Програма порівнює довжину кожного рядка з вирахованим максимумом і збирає всі відповідні результати в окремий список. Такий підхід гарантує, що ми побачимо всі найдовші рядки, а не лише перший, що зустрівся у файлі.

Особливу увагу я приділила обробці станів файлу. Якщо файл виявляється абсолютно порожнім або містить лише пробіли та переноси

рядків, програма не просто «мовчить», а виводить зрозуміле повідомлення про те, що слів для аналізу не знайдено. Також я додала блок обробки виключень try-catch, щоб уникнути аварійного завершення, якщо сам файл input.txt раптом буде відсутній у директорії проєкту. Сам текстовий файл я розмістила в кореневій папці, щоб програма могла звертатися до нього за відносним шляхом без додаткових налаштувань.

Скріншоти коду

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.List;
import java.util.stream.Collectors;

public class MaxWordsFinder {
    public static void main(String[] args) {
        String filePath = "input.txt";

        try {
            List<String> allLines = Files.readAllLines(Paths.get(filePath));

            // Знаходимо максимальну кількість слів серед усіх рядків
            int maxWords = allLines.stream()
                .mapToInt(line -> line.trim().isEmpty() ? 0 : line.trim().split("\\s+").length)
                .max()
                .orElse(0);

            // Якщо слів взагалі немає (maxWords == 0), просто виводимо повідомлення
            if (maxWords == 0) {
                System.out.println("Файл порожній або не містить слів.");
                return;
            }

            // Знаходимо всі рядки з такою кількістю слів
            List<String> resultLines = allLines.stream()
                .filter(line -> !line.trim().isEmpty() && line.trim().split("\\s+").length == maxWords)
                .collect(Collectors.toList());

            System.out.println("Максимальна кількість слів: " + maxWords);
            resultLines.forEach(System.out::println);

        } catch (IOException e) {
            System.err.println("Помилка читання: " + e.getMessage());
        }
    }
}
```

Скріншоти виконання коду для порожнього файлу

lab_5_1 >  input.txt

1 Use Cmd + I to generate code

java MaxWordsFinder.java

Файл порожній або не містить слів.

Скріншоти виконання коду для файлу з кількома рядками однакової довжини

```
lab_5_1 >  input.txt
1   Гуцулко Ксеню,
2   Я тобі на трембіті
3
4   Розкажу про любов.
5   Душа страждає,
6   Звук трембіти лунає,
7   Та що серце кохає,
8   Бо гаряче, мов жар
9   |
```

```
java MaxWordsFinder.java
Максимальна кількість слів: 4
Я тобі на трембіті
Та що серце кохає,
Бо гаряче, мов жар
```

Скріншоти виконання коду для файлу з одним рядком максимальної довжини

```
lab_5_1 >  input.txt
1   Гуцулко Ксеню,
2   Я тобі на трембіті
3   Лиш одній в цілім світі
4   Розкажу про любов.
5   Душа страждає,
6   Звук трембіти лунає,
7   Та що серце кохає,
8   Бо гаряче, мов жар
9   |
```

```
java MaxWordsFinder.java
Максимальна кількість слів: 5
Лиш одній в цілім світі
```

Вирішення завдання (Завдання 2)

У межах цього завдання я реалізувала алгоритм шифрування Цезаря, використовуючи механізм фільтрації символьних потоків. Основною вимогою було застосування класів типу `FilterWriter` та `FilterReader`, що дозволяє виконувати трансформацію даних безпосередньо в процесі їхнього запису або зчитування. Для цього я розробила два власні класи — `CaesarWriter` та `CaesarReader`.

Логіка шифрування полягає у зміщенні числового коду кожного символу на значення коду обраного ключа. В класі `CaesarWriter` я перевизначила метод `write`, який перед відправкою символу в основний потік додає до нього значення ключа. Дешифрування реалізовано дзеркально в класі `CaesarReader`: під час читання метод `read` автоматично віднімає значення ключа, повертаючи початковий символ. Такий підхід робить процес шифрування "прозорим" для основної програми, оскільки вона працює зі звичайними методами читання та запису, не вникаючи в деталі кодування.

Під час тестування я звернула увагу на технічні особливості ієрархії класів `java.io`. Зокрема, для коректної обробки поодиноких символів у `CaesarReader` я реалізувала пряме звернення до підстилаючого потоку `in`. Це забезпечило стабільну роботу програми при читанні файлів до самого кінця (символ `-1`). Програма успішно записує зашифрований "шум" у файл `encrypted.txt` і миттєво відновлює оригінальний текст при повторному зчитуванні, що підтверджує правильність обраної архітектури.

```

1  import java.io.*;
2
3  public class CaesarCipherApp {
4      public static void main(String[] args) {
5          // Текст для перевірки
6          String originalText = "This is text for testing Caesar algorythm";
7          char key = (char) 5;
8          String filename = "encrypted.txt";
9
10         // а) Шифрування
11         try (CaesarWriter cw = new CaesarWriter(new FileWriter(filename), key)) {
12             cw.write(originalText);
13             System.out.println("Текст зашифровано у файл: " + filename);
14         } catch (IOException e) {
15             System.err.println("Помилка запису: " + e.getMessage());
16         }
17
18         // б) Дешифрування
19         System.out.print("Дешифрований текст: ");
20         try (CaesarReader cr = new CaesarReader(new FileReader(filename), key)) {
21             int data;
22             while ((data = cr.read()) != -1) {
23                 System.out.print((char) data);
24             }
25             System.out.println();
26         } catch (IOException e) {
27             System.err.println("Помилка читання: " + e.getMessage());
28         }
29     }
30 }
31

```

```

31
32 // Клас для шифрування
33 class CaesarWriter extends FilterWriter {
34     private final char key;
35
36     public CaesarWriter(Writer out, char key) {
37         super(out);
38         this.key = key;
39     }
40
41     // Шифруємо кожен символ перед записом
42     @Override
43     public void write(int c) throws IOException {
44         out.write(c + key);
45     }
46
47     // Додатковий метод для запису цілих рядків
48     @Override
49     public void write(String str, int off, int len) throws IOException {
50         for (int i = 0; i < len; i++) {
51             write(str.charAt(off + i));
52         }
53     }
54 }
55

```

```

56 // Клас для дешифрування
57 class CaesarReader extends FilterReader {
58     private final char key;
59
60     public CaesarReader(Reader in, char key) {
61         super(in);
62         this.key = key;
63     }
64
65     // Дешифруємо символ під час читання
66     @Override
67     public int read() throws IOException {
68         int c = in.read();
69         if (c == -1) return -1;
70         return c - key;
71     }
72 }

```

Скріншоти виконання коду

```

Текст зашифровано у файл: encrypted.txt
Дешифрований текст: This is text for testing Caesar algorithnm

```

Вирішення завдання (Завдання 3)

Для виконання цього завдання я вирішила використовувати виключно стандартні можливості Java, щоб забезпечити максимальну сумісність програми без підключення сторонніх залежностей. Робота алгоритму побудована на взаємодії з мережевими потоками та використанні регулярних виразів для парсингу HTML-структури.

Логіка реалізації:

- Отримання даних: Програма створює підключення до вказаного URL за допомогою класу URL та зчитує весь зміст сторінки у StringBuilder через BufferedReader. Це дозволяє отримати повний HTML-код сторінки як один великий рядок.

- Парсинг тегів: Оскільки HTML має чітку структуру, я використала регулярний вираз `<\s*([a-zA-Z0-9]+)`, який знаходить відкриваючі теги та витягує їхні назви. Знайдені назви приводяться до нижнього регістру для уникнення дублювання (наприклад, `<div>` та `<DIV>`).

- Обробка статистики: Всі знайдені теги зберігаються у HashMap. Для виконання вимог завдання я використала два різні підходи до сортування: для лексикографічного порядку дані копіюються в TreeMap, яка автоматично впорядковує ключі за алфавітом, для сортування за частотою використано Stream API з компаратором `comparingByValue()`, що дозволяє вивести результати від найменш вживаних тегів до найбільш популярних.

```

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.URL;
import java.util.*;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class TagFrequencyCounter {
    public static void main(String[] args) {
        String urlString = "https://www.google.com";

        try {
            URL url = new URL(urlString);
            BufferedReader reader = new BufferedReader(new InputStreamReader(url.openStream()));

            StringBuilder content = new StringBuilder();
            String line;
            while ((line = reader.readLine()) != null) {
                content.append(line);
            }
            reader.close();

            // Регулярний вираз для пошуку назв тегів
            // Шукає текст відразу після '<' (наприклад, <div -> div)
            Pattern pattern = Pattern.compile("<\\s*([a-zA-Z0-9]+)");
            Matcher matcher = pattern.matcher(content.toString());

            Map<String, Integer> tagMap = new HashMap<>();
            while (matcher.find()) {
                String tagName = matcher.group(1).toLowerCase();
                tagMap.put(tagName, tagMap.getOrDefault(tagName, 0) + 1);
            }

            if (tagMap.isEmpty()) {
                System.out.println("Тегів не знайдено або доступ заблоковано.");
                return;
            }

            // a) В лексикографічному порядку
            System.out.println("--- Теги в лексикографічному порядку ---");
            new TreeMap<>(tagMap).forEach((tag, count) -> System.out.println(tag + ": " + count));

            // b) В порядку зростання частоти
            System.out.println("\n--- Теги за частотою (зростання) ---");
            tagMap.entrySet().stream()
                .sorted(Map.Entry.comparingByValue())
                .forEach(entry -> System.out.println(entry.getKey() + ": " + entry.getValue()));

        } catch (Exception e) {
            System.err.println("Помилка: " + e.getMessage());
        }
    }
}

```

Скріншоти виконання коду


```
--- Теги в лексикографічному порядку ---
0: 1
a: 18
b: 1
body: 1
br: 5
c: 1
center: 1
div: 11
form: 1
head: 1
html: 1
img: 2
input: 10
meta: 2
nobr: 2
p: 1
script: 7
span: 8
style: 2
table: 1
td: 3
title: 1
tr: 1
u: 1

--- Теги за частотою (зростання) ---
b: 1
c: 1
center: 1
title: 1
body: 1
head: 1
0: 1
p: 1
form: 1
u: 1
html: 1
table: 1
tr: 1
nobr: 2
img: 2
meta: 2
style: 2
td: 3
br: 5
script: 7
span: 8
input: 10
div: 11
a: 18
```

Висновки

Під час виконання лабораторної роботи №5 я поглибила свої знання у роботі з фундаментальними механізмами мови Java, зокрема у сферах обробки текстової інформації, взаємодії з файловими системами та мережевими протоколами.

Робота над першим завданням дозволила мені закріпити навички використання Java Stream API для аналізу текстових даних. Я переконалася, що такий підхід значно спрощує обробку великих масивів інформації та дозволяє лаконічно реалізовувати складну логіку фільтрації.

Друге завдання стало корисним досвідом у проєктуванні власних обробників потоків. Реалізація шифру Цезаря через механізм FilterWriter та FilterReader продемонструвала переваги об'єктно-орієнтованого підходу:

можливість розділити логіку трансформації даних від безпосереднього процесу введення-виведення.

Особливий інтерес викликало третє завдання, де я навчилася взаємодіяти з веб-ресурсами за допомогою стандартних засобів Java. Використання регулярних виразів для парсингу HTML-структур та застосування різних типів колекцій, таких як TreeMap та HashMap, дало змогу ефективно згрупувати та відсортувати статистичні дані за різними критеріями.

Загалом, виконання цієї роботи навчило мене створювати стійкі до помилок додатки, які коректно обробляють порожні файли, відсутність мережевого з'єднання або нестандартні символи вхідних даних. Ці навички є базовими для розробки сучасного програмного забезпечення на мові Java.